Kalahari Magnetic and Soil Survey Node

A low-cost, GPS-tagged embedded system for reconnaissance mining exploration

Embedded systems design project | Montle Fredah Segomotso | Gaborone, Botswana | October 2026

Project summary
A handheld survey staff built around an ESP32. At each station the operator presses one button; the node measures the local magnetic field, soil resistance and soil temperature, tags the reading with a GPS position and logs it to a microSD card. A Python script turns the log into anomaly maps so that a small exploration team can decide where to look more closely, before paying for professional geophysics.

Status: design, firmware, simulation files and printable parts are complete. A physical build and field calibration are the next steps. All numbers in the results figure are synthetic and illustrative.

Contents

- 1. Problem statement and objectives
- 2. Methodology
- 3. System architecture
- 4. Circuit schematics and pin map
- 5. Simulation (Wokwi)
- 6. 3D model prototype
- 7. Data processing and example output
- 8. Cost of materials (AliExpress) and component list
- 9. Test plan, limitations and next steps
- Appendix A: firmware | B: Wokwi diagram.json | C: Python mapping script

1. Problem statement

Mineral exploration in Botswana often takes place where the Kalahari sand cover hides the rock underneath. Features of interest, such as kimberlite pipes that may host diamonds, or magnetic and conductive sulphide bodies, can leave only faint signatures at the surface: a small change in the Earth's magnetic field, or a change in how well the soil conducts electricity.

Professional ground magnetometers and conductivity meters can detect these signatures, but they are expensive and are usually operated by specialist crews. Small exploration teams, students and community-level prospectors often rely on handwritten notes, a handheld GPS and sample bags. This leads to three practical problems:

- **Cost:** blanket coverage of a licence area with commercial instruments is unaffordable for small teams, so large areas are never screened.
- **Data quality:** manual recording of position and readings causes transcription errors and makes later mapping slow.
- **Single-parameter views:** a magnetic reading on its own is ambiguous. Pairing it with soil conductivity and temperature at the same point helps decide which anomalies deserve follow-up.

Problem to solve: design an affordable, field-portable instrument that records magnetic field, soil resistance and soil temperature together with an accurate position at every survey station, stores the data without manual typing, and produces maps that point a geologist to the stations most worth investigating.

1.1 Objectives

- Record |B|, soil resistance (converted to a conductance index), soil temperature and GPS position per station in one button press.
- Log to a plain CSV file on microSD that opens in any spreadsheet or Python tool.
- Run for a full field day on one 18650 cell and survive dust and heat, with a printable enclosure.
- Keep the bill of materials near USD 50, using parts available on AliExpress.
- Provide a simulation, firmware and processing script so the design can be tested before building.

1.2 Scope and honest limits

This is a **reconnaissance screening tool**, not a replacement for survey-grade equipment. The QMC5883L is a hobby-grade sensor. Its noise is typically in the range of tens to hundreds of nT, against an Earth field of roughly 30,000 nT in this region, so it can reveal only strong or shallow magnetic contrasts and works best as a relative (anomaly) measurement. Two-electrode soil resistance is affected by moisture, salinity and electrode contact. The firmware and processing steps below are designed to reduce these effects, and the test plan in Section 9 measures them, but results must always be confirmed with proper geophysics and sampling.

2. Methodology

2.1 What is measured and why

Measurement	Sensor	Why it matters
Total magnetic field |B|	QMC5883L, 3-axis, I2C	Magnetic minerals (for example magnetite-bearing kimberlite or pyrrhotite) distort the local field. |B| = sqrt(Bx^2 + By^2 + Bz^2) is much less sensitive to how the sensor is tilted than a single axis.
Soil resistance Rs	Two stainless electrodes, 10 kOhm reference, ESP32 ADC	Conductive bodies and mineralised or saline zones lower the resistance. Reported as a conductance index (1,000,000 / Rs, in µS) and corrected to 25 degC.
Soil temperature	DS18B20 probe	Conductivity changes about 2 % per degC, so temperature is needed to compare stations measured at different times of day.
Position and time	NEO-6M GPS	Places every reading on a map. Stations with HDOP above 3 are flagged as not fixed.

2.2 Field procedure

- Lay out a grid, for example 10 m station spacing on lines 10 m apart, with lines across the expected geological strike.

- Carry the instrument on a non-magnetic pole. The magnetometer pod sits at the top of the pole, about 0.8 m from the electronics enclosure, to keep battery and switching currents away from it.
- Hold the pole vertical with the pod facing the same compass direction at every station, and keep watches, phones, belts and steel boots away from the sensor.
- At each station push the electrode fork into the soil, wait for the OLED to show a GPS fix, then press the button. One short beep means saved with fix; three beeps means saved without a fix or an SD error; a long beep means the magnetometer failed.
- Return to a fixed base point every 20 to 30 minutes and log a reading. Drift at the base point shows how much the field changed with time, so it can be subtracted later.
- Copy survey.csv from the card and run the mapping script.

2.3 Measurement and signal processing

- **Magnetometer:** 20 samples at 200 Hz output rate with 512x oversampling are averaged at each station. One LSB is about 33.3 nT in the 8 gauss range.
- **Soil resistance:** the ESP32 drives the electrodes through a 10 kOhm reference resistor with alternating polarity (2 ms each way, repeated 8 times) and releases both pins in between, so no DC builds up across the soil. Rs is computed from the divider voltage at node N1 (Section 4.2). Readings outside 0.15 to 3.1 V are discarded because the ESP32 ADC is non-linear near the rails; a dry-sand open circuit is therefore reported as no reading rather than a wrong number.
- **Temperature correction:** EC25 = EC / (1 + 0.02 x (T - 25)).
- **Mapping:** positions are converted to local metres. A best-fit plane is subtracted from |B| to remove the regional gradient, leaving the anomaly. Stations more than 2 standard deviations from the mean magnetic anomaly, or with high conductance, are flagged and listed in anomalies.csv.

2.4 Design and verification workflow

- Define the problem and the measurements (this section).
- Design the circuits and pin map, then draw the schematics (Section 4).
- Simulate in Wokwi with potentiometers standing in for the sensors that Wokwi does not provide (Section 5).
- Model the enclosure, magnetometer pod and electrode fork for 3D printing (Section 6).
- Build on perfboard, then run the bench and field tests in Section 9.

3. System architecture

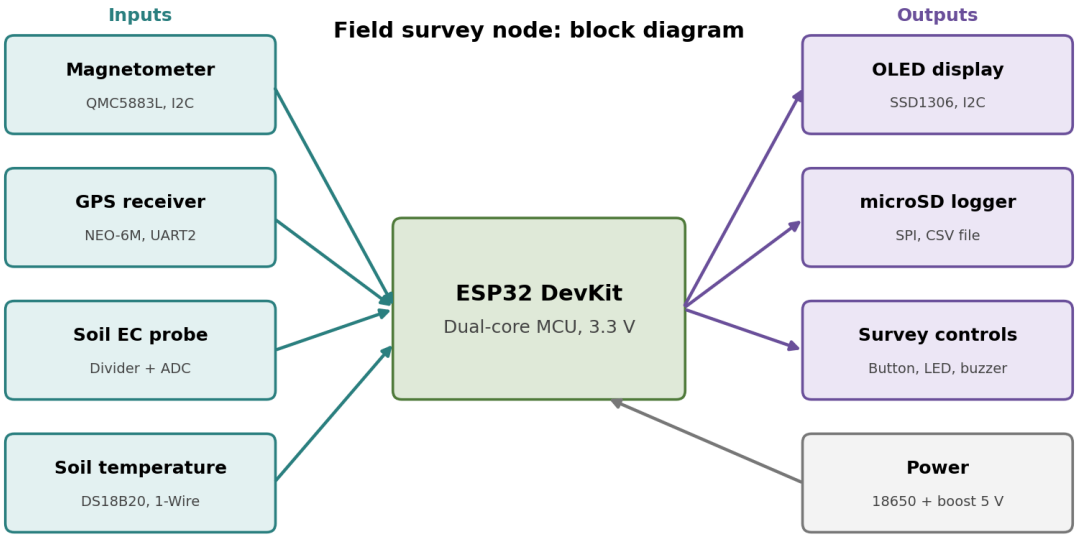


Figure 1. Block diagram. The ESP32 reads four sensing blocks and writes to the display, SD card and indicators.

Operation sequence on each button press: debounce, read GPS position and time, average the magnetometer, measure soil resistance with alternating polarity, read the DS18B20, compute conductance and its 25 degC value, append one CSV row, show the result on the OLED and sound the buzzer. The CSV columns are:

```
station,utc,lat,lon,hdop,sats,bx_nT,by_nT,bz_nT,btotal_nT,rs_ohm,ec_uS,temp_C,ec25_uS,fix_ok
```

4. Circuit schematics and pin map

4.1 Main interconnect

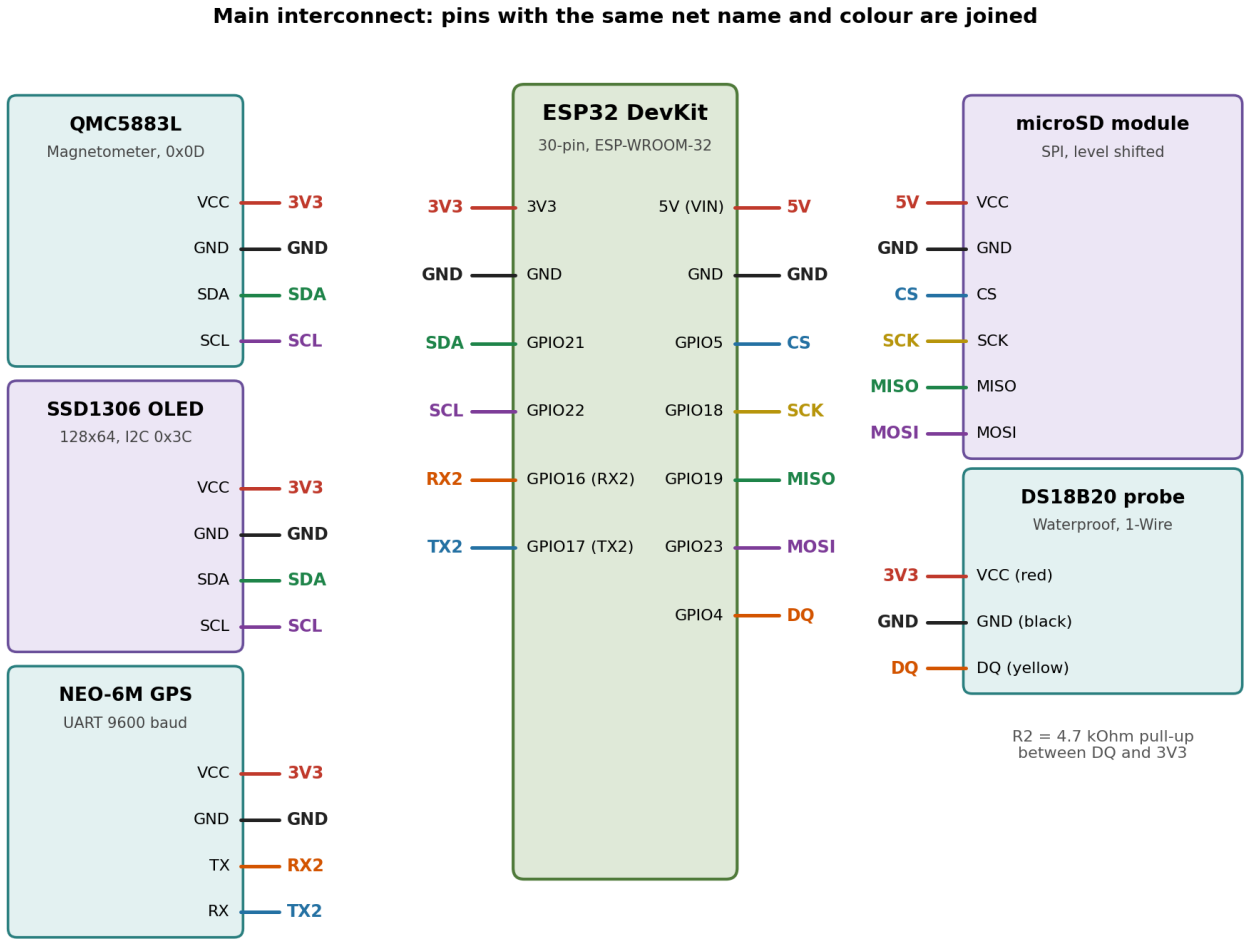


Figure 2. Main interconnect. The I2C bus (GPIO21, GPIO22) is shared by the magnetometer and OLED. Use a microSD module that has its own level shifter and regulator (the common 5 V type).

Pin map

Signal	Component pin	ESP32 pin	Notes
I2C SDA	QMC5883L SDA, OLED SDA	GPIO21	Addresses 0x0D (QMC5883L) and 0x3C (OLED)
I2C SCL	QMC5883L SCL, OLED SCL	GPIO22	100 kHz bus clock
GPS RX2	NEO-6M TX	GPIO16	9600 baud, UART2
GPS TX2	NEO-6M RX	GPIO17	
SD CS	microSD CS	GPIO5	SPI chip select
SD SCK / MISO / MOSI	microSD SCK / MISO / MOSI	GPIO18 / 19 / 23	Default VSPI pins
1-Wire	DS18B20 DQ	GPIO4	4.7 kOhm pull-up to 3V3 (R2)
EC drive A	R1 (10 kOhm), electrode A	GPIO25	Reference side
EC drive B	Electrode B	GPIO26	Direct to probe
EC sense	Node N1 via R5 (1 kOhm)	GPIO34	Input-only ADC pin

Signal	Component pin	ESP32 pin	Notes
Button	SW1 to GND	GPIO27	Internal pull-up
Status LED	D1 via R3 (220 Ohm)	GPIO2	Active high
Buzzer	Q1 base via R4 (1 kOhm)	GPIO13	Active high
Simulation only	Magnetometer stand-in pot	GPIO35	Unused on real hardware
Power	MT3608 5.0 V output	5V (VIN) and GND	Board regulator makes 3V3

4.2 Soil resistance front end

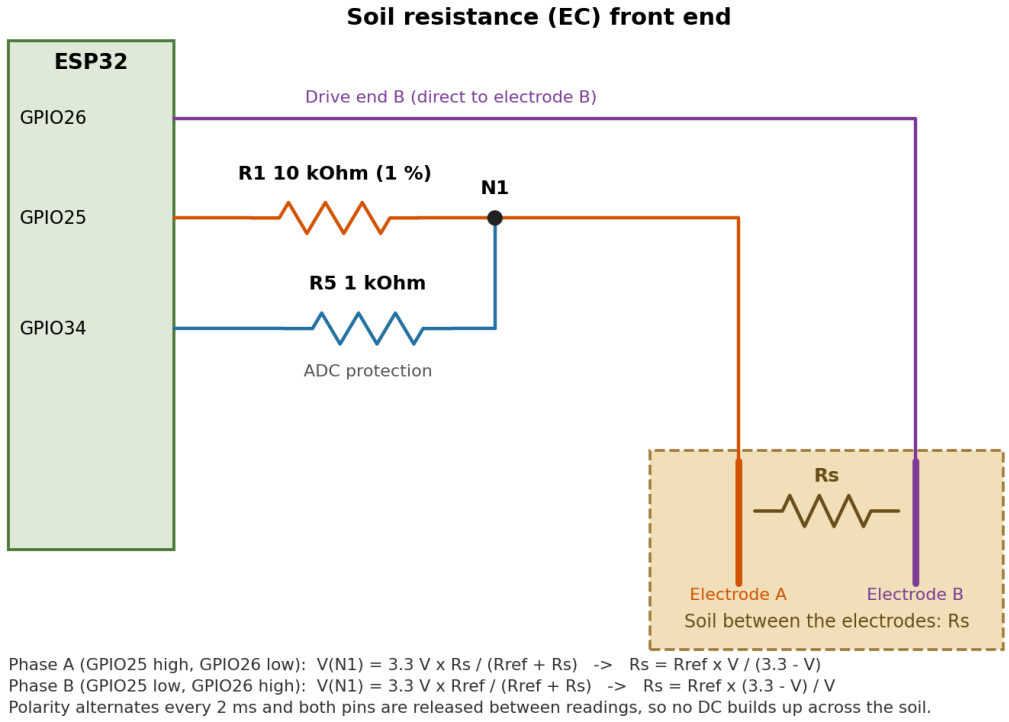


Figure 3. Two-electrode soil resistance circuit with alternating polarity.

In phase A, GPIO25 is high and GPIO26 low, so V(N1) = 3.3 V x Rs / (Rref + Rs) and Rs = Rref x V / (3.3 - V). In phase B the drive is reversed, V(N1) = 3.3 V x Rref / (Rref + Rs) and Rs = Rref x (3.3 - V) / V. With Rref = 10 kOhm the circuit works well for Rs between roughly 2 kOhm and 100 kOhm. Use 4 mm 316 or 304 stainless rods, 50 mm apart. The reported conductance index is relative; to turn it into true conductivity, measure a standard salt solution and find the cell constant (Section 9).

4.3 Controls, buzzer and temperature probe

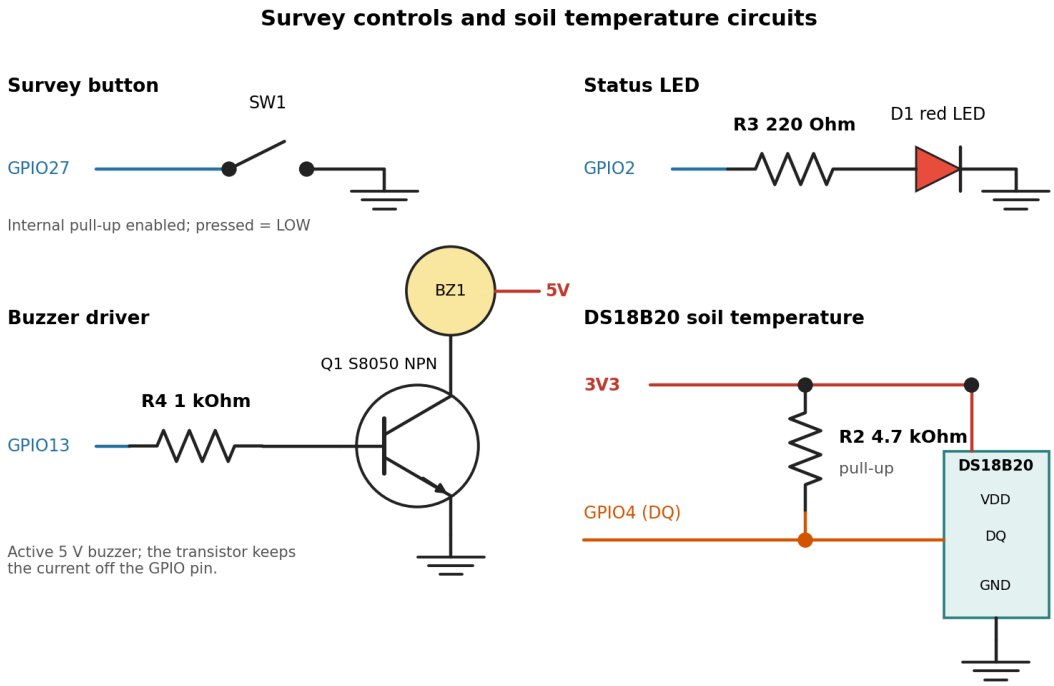


Figure 4. Button, LED, transistor-driven buzzer and DS18B20 circuit.

4.4 Power

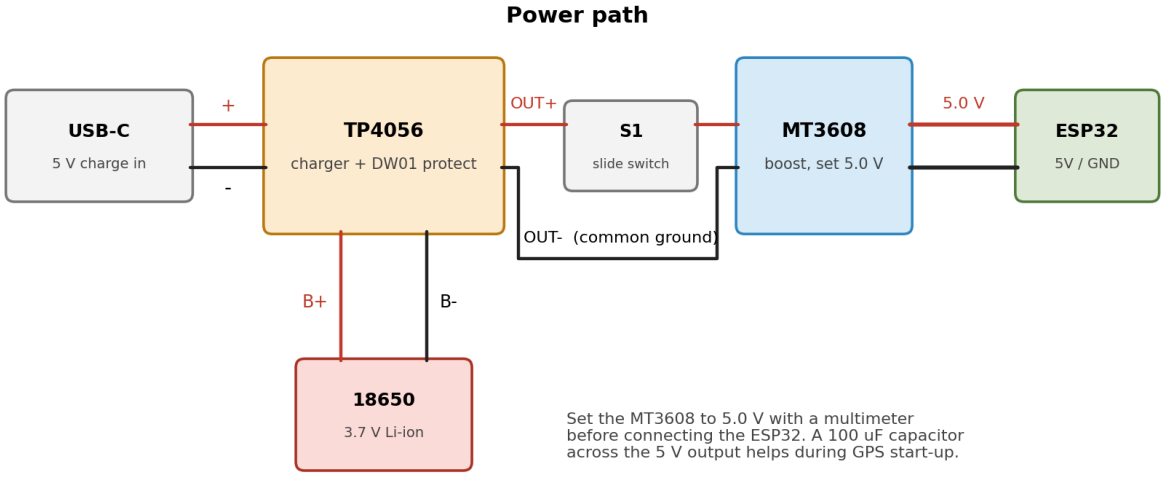


Figure 5. One 18650 cell feeds a TP4056 charger board (with protection) and an MT3608 boost converter set to 5.0 V. The ESP32 board makes 3.3 V for the sensors. Average current is roughly 100 to 150 mA, so a 2600 mAh cell should run a field day; the test plan measures this.

5. Simulation (Wokwi)

The Wokwi project files are supplied in the simulation folder (diagram.json, libraries.txt, README_wokwi.txt). Set `#define SIM_MODE 1` in the sketch before running. Three parts are replaced by stand-ins:

Real part	Simulation stand-in	How to use it
QMC5883L magnetometer	Potentiometer on GPIO35	Turn the knob: field varies from 25,000 to 35,000 nT
Soil probe divider (node N1)	Potentiometer on GPIO34	Knob sets V(N1); firmware converts to Rs
NEO-6M GPS	Fake walking track near Gaborone	Position advances about 10 m per press

Steps: create an ESP32 project at wokwi.com, replace diagram.json and libraries.txt, paste the firmware as sketch.ino with SIM_MODE set to 1, press Play, turn the two knobs and press the green button. Each press prints a line to the Serial Monitor, updates the OLED and writes a row to survey.csv on the simulated SD card. The part and pin names in diagram.json follow Wokwi's catalogue as best known; if Wokwi flags a name, re-add that part from the + menu and wire it according to the pin map in Section 4.

The same circuit can be drawn in Proteus or KiCad using the schematics and component list in this document; the net-label schematic in Figure 2 maps directly onto a KiCad sheet.

6. 3D model prototype

Five printable parts were modelled in millimetres and exported as STL files (enclosure folder). Overlapping solids are used inside some parts, so run each file through your slicer's repair or union step before printing.

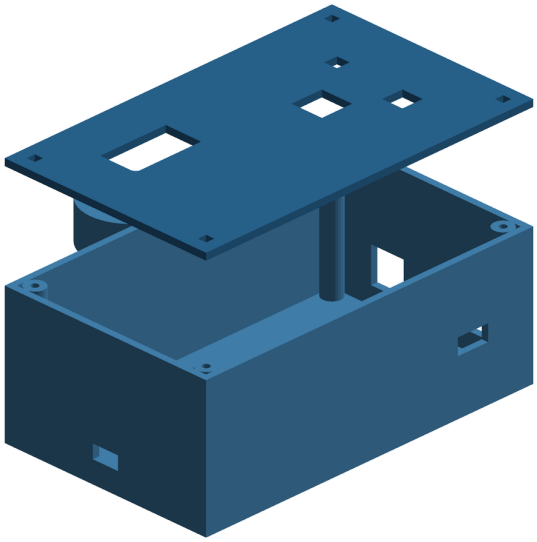


Figure 6. Electronics enclosure, 130 x 80 x 50 mm, with lid lifted. The lid has an OLED window, button hole, LED hole and buzzer vent. The back carries two ring clamps for a 20 mm pole.

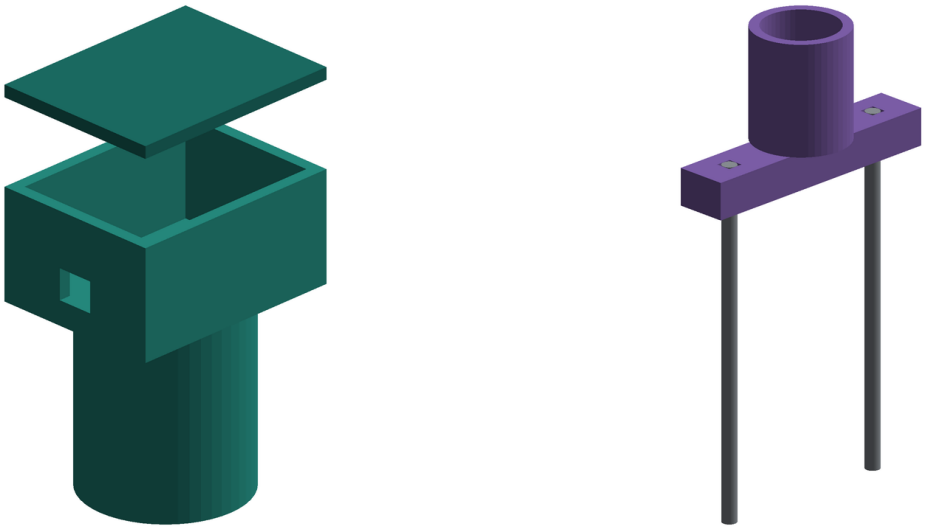


Figure 7. Magnetometer pod (left, exploded) and electrode fork with two 4 mm rods (right).



Figure 8. Full staff, 1.9 m: fork at the bottom, enclosure at about 1.0 m, pod at the top.

File	Part	Size (mm)	Notes
enclosure_body.stl	Enclosure body	130 x 80 x 50	2.5 mm walls, four M3 posts, side openings for cable glands, USB-C slot and switch, two clamp rings
enclosure_lid.stl	Enclosure lid	130 x 80 x 2.5	Print top face down. Square openings are placeholders; the screw holes are 3.2 mm
magnetometer_pod.stl	Sensor pod with pole socket	36 x 28 x 18 + socket	Socket fits a 20 mm pipe; cable slot on one end
magnetometer_pod_lid.stl	Pod lid	36 x 28 x 2	Fix with nylon screws only
ec_probe_fork.stl	Electrode spacer and socket	70 x 14 x 10 + socket	Two 4.2 mm square seats, 50 mm apart; glue the rods in with epoxy

Print in PETG (better heat tolerance than PLA in a vehicle or in the sun), 0.2 mm layers, three perimeters, about 30 % infill. Estimated filament use is roughly 90 to 130 g for all five parts. Use only non-magnetic fasteners (nylon or brass) near the magnetometer. The pole should be PVC, fibreglass or wood, never steel. Because the sensor sits about 0.8 m above the electronics, measure the noise with the electronics powered on and off during testing; if the difference is significant, lengthen the boom.

7. Data processing and example output

map_survey.py reads survey.csv, keeps stations with a GPS fix, removes the regional gradient from the magnetic data, temperature-corrects the conductance, flags outliers and draws two maps. Running `python map_survey.py --demo` creates a synthetic survey with one buried magnetic and conductive body so the pipeline can be tested without hardware.

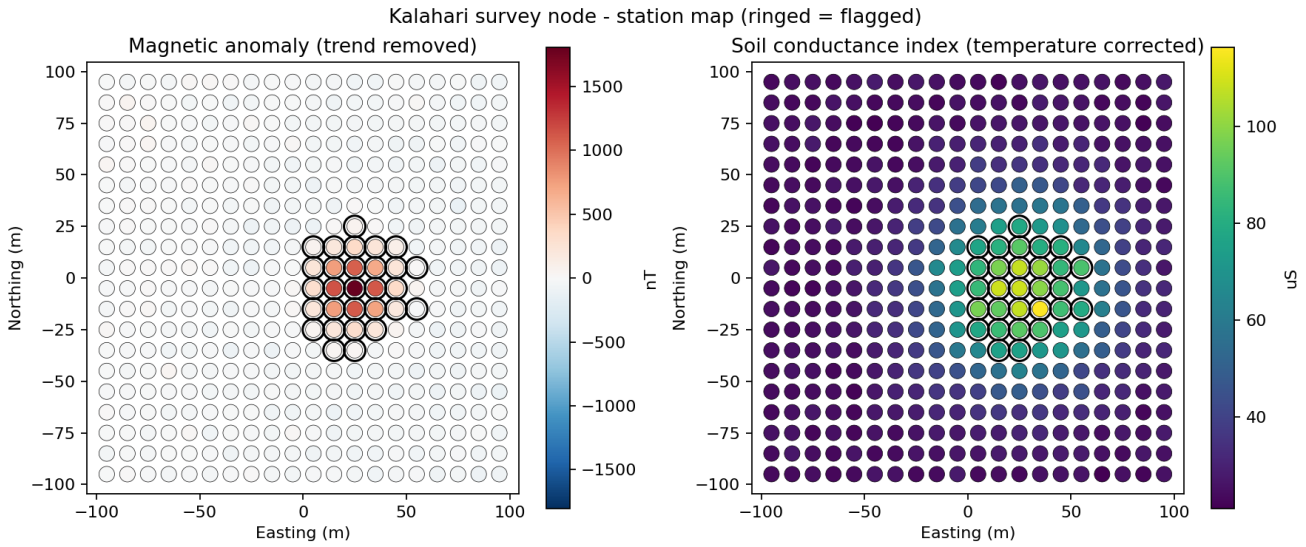


Figure 9. Output of the mapping script on synthetic data (not a real survey). Ringed stations are the flagged anomalies. In a real survey, ringed clusters mark where to run a finer 5 m grid and where a geologist should sample.

8. Cost of materials and component list

8.1 Bill of materials (approximate AliExpress prices)

Please read: these are estimates for typical single-unit listings. I could not look up live AliExpress prices while preparing this document, so confirm each price at checkout. Shipping to Botswana, import duty and VAT are not included, and lithium cells are often not shipped to individuals, which is why the battery and the SD card are marked for local purchase.

#	Item	Qty	Unit (USD)	Subtotal	Notes
1	ESP32 DevKit (ESP-WROOM-32, 30-pin)	1	4.50	4.50	AliExpress
2	QMC5883L magnetometer module (GY-271 style)	1	1.50	1.50	AliExpress. Check the listing says QMC5883L, not HMC5883L
3	NEO-6M GPS module with ceramic antenna	1	4.50	4.50	AliExpress
4	0.96 in SSD1306 OLED, 128x64, I2C	1	2.00	2.00	AliExpress
5	microSD SPI module with level shifter	1	1.00	1.00	AliExpress
6	microSD card, 16 GB	1	4.00	4.00	Buy locally or AliExpress
7	DS18B20 waterproof probe, 1 m cable	1	1.60	1.60	AliExpress
8	TP4056 USB-C charger board with protection	1	0.80	0.80	AliExpress
9	MT3608 boost converter module	1	0.80	0.80	AliExpress
10	18650 Li-ion cell, protected, 2600 mAh	1	3.50	3.50	Often restricted for shipping; buy locally
11	18650 single-cell holder with leads	1	0.80	0.80	AliExpress
12	Slide switch (SS12D00 type), pack of 2	1	0.50	0.50	AliExpress
13	12 mm waterproof push button	1	1.00	1.00	AliExpress
14	Active buzzer, 5 V	1	0.60	0.60	AliExpress
15	LED 5 mm red (pack)	1	0.30	0.30	AliExpress
16	S8050 NPN transistor (pack)	1	0.50	0.50	AliExpress
17	Resistor kit, 1 % metal film (10k, 4.7k, 1k, 220 Ohm)	1	2.50	2.50	AliExpress. Use 1 % for R1
18	100 uF 16 V electrolytic capacitor (pack)	1	0.80	0.80	AliExpress
19	Stainless rod 4 mm x 150 mm (304 or 316)	2	2.20	4.40	AliExpress or local metal shop
20	PG7 cable glands	2	0.50	1.00	AliExpress
21	Nylon M3 screw and standoff assortment	1	3.00	3.00	AliExpress. Non-magnetic
22	Perfboard 7x9 cm, headers, screw terminals	1	3.00	3.00	AliExpress
23	Silicone wire and Dupont jumper set	1	2.50	2.50	AliExpress
24	PETG filament, about 120 g used (spool about USD 15)	1	1.80	1.80	Prorated; buy a 1 kg spool or ask a print shop
25	PVC or fibreglass pole, 20 mm, about 1.9 m	1	4.00	4.00	Local hardware store, not AliExpress
	Subtotal			**50.90**	
	Contingency 10 % (price changes, spares)			5.09	
	Estimated total, before shipping and tax			**55.99**	

At an assumed rate of P13.5 per USD (check the current rate), the estimated total is about P756. The items that most affect quality are the magnetometer module (clone boards vary), the GPS antenna and the 1 % reference resistor.

8.2 Circuit components and designators

Ref	Component	Value / part	Qty	Connected to
U1	Microcontroller	ESP32 DevKit, ESP-WROOM-32	1	See pin map
U2	Magnetometer	QMC5883L (GY-271 board)	1	3V3, GND, SDA GPIO21, SCL GPIO22
U3	GPS	NEO-6M with antenna	1	3V3, GND, TX to GPIO16, RX to GPIO17
U4	OLED	SSD1306 128x64 I2C, 0x3C	1	3V3, GND, SDA, SCL

Ref	Component	Value / part	Qty	Connected to
U5	microSD module	SPI, 5 V with level shifter	1	5V, GND, GPIO5/18/19/23
U6	Temperature probe	DS18B20 waterproof	1	3V3, GND, GPIO4
U7	Charger	TP4056 USB-C with DW01	1	USB-C in, B+/B- to cell, OUT+/OUT-
U8	Boost converter	MT3608, set to 5.0 V	1	From S1 and OUT-; to ESP32 5V and GND
BT1	Battery	18650 Li-ion 3.7 V	1	TP4056 B+/B-
S1	Power switch	Slide switch	1	TP4056 OUT+ to MT3608 IN+
SW1	Survey button	12 mm tactile	1	GPIO27 to GND
D1	Status LED	5 mm red	1	GPIO2 via R3 to GND
BZ1	Buzzer	Active 5 V	1	5V to Q1 collector
Q1	Buzzer driver	S8050 NPN	1	Base via R4 from GPIO13, emitter GND
R1	EC reference resistor	10 kOhm, 1 %	1	GPIO25 to node N1
R2	1-Wire pull-up	4.7 kOhm	1	DQ to 3V3
R3	LED resistor	220 Ohm	1	GPIO2 to D1
R4	Transistor base resistor	1 kOhm	1	GPIO13 to Q1 base
R5	ADC protection resistor	1 kOhm	1	Node N1 to GPIO34
C1	Bulk capacitor	100 µF, 16 V	1	Across 5 V output (observe polarity)
E1, E2	Soil electrodes	Stainless rod 4 mm x 150 mm	2	E1 to N1, E2 to GPIO26

9. Test plan, limitations and next steps

9.1 Bench and field tests

Test	How	Pass criterion
Magnetometer noise	Log 100 readings with the sensor still, electronics on, then off	Standard deviation recorded; the on/off difference tells you whether the boom is long enough
Magnetometer response	Move a small magnet at set distances from the pod	Reading changes smoothly and repeatably; note the detection distance
Heading dependence	Rotate the pod through 360 degrees in 45 degree steps	Spread of |B| recorded; decide whether a calibration or fixed heading is enough
EC calibration	Measure salt solutions of known conductivity (for example 1413 µS/cm standard)	Cell constant K found so that EC = K / Rs; repeat at two temperatures
Repeatability	Measure the same station five times, re-inserting the fork each time	Spread of Rs within an acceptable band, to be set after the first trials
GPS	Compare logged positions against a known benchmark	Typical error within a few metres in open sky
Battery life	Log a station every minute until the cell is flat	At least one field day
Dust and heat	Leave the closed enclosure in the sun and check the OLED and SD card	No resets or write errors

9.2 Known limitations

- The QMC5883L is not survey-grade and its readings can drift with temperature; the base-point readings and the plane-fit removal reduce, but do not eliminate, this.
- Soil resistance by two electrodes includes contact resistance and depends on moisture. Treat it as a relative index until calibrated, and avoid comparing stations measured shortly after rain with dry ones.
- Dry Kalahari sand can be too resistive for the reference resistor; the firmware then reports an open circuit. Wetting the electrode spot or using a larger reference resistor are options to try.
- The enclosure parts are untested designs and the square openings are placeholders for glands and the USB-C port; expect one print iteration.
- A flagged anomaly is a reason to investigate, not evidence of a deposit.

9.3 Next steps

- Run the Wokwi simulation and fix any part or pin name it flags.
- Build the circuit on perfboard and run the bench tests above.
- Print the five parts and assemble the staff.
- Do a first test survey on a small grid with a known magnetic feature (for example a buried steel plate), then calibrate and adjust thresholds.
- Add a second unit or a regular base-station routine for diurnal correction.

Appendix A: firmware (kalahari_survey_node.ino)

Arduino IDE with the ESP32 board package. Libraries: Adafruit SSD1306, Adafruit GFX Library, TinyGPSPlus, OneWire, DallasTemperature. Set SIM_MODE to 0 for real hardware.

```
/*
  Kalahari Magnetic and Soil Survey Node
  Board : ESP32 DevKit (ESP-WROOM-32)
  Author: Montle Fredah Segomotso

  Press the survey button at each station. The node averages the magnetometer,
  measures soil resistance with alternating polarity, reads soil temperature,
  and appends one CSV row (with GPS position) to the microSD card.

  Libraries (Arduino Library Manager):
    Adafruit SSD1306, Adafruit GFX Library, TinyGPSPlus,
    OneWire, DallasTemperature   (SD and SPI come with the ESP32 core)

  SIM_MODE = 1 -> Wokwi simulation:
    GPIO35 potentiometer stands in for the magnetometer,
    GPIO34 potentiometer stands in for the soil divider node (N1),
    and a fake walking track replaces the GPS.
  SIM_MODE = 0 -> real hardware.
*/

#define SIM_MODE 0

#include <Wire.h>
#include <SPI.h>
#include <SD.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <TinyGPSPlus.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// ---------- Pin map ----------
#define PIN_SDA      21
#define PIN_SCL      22
#define PIN_GPS_RX   16   // ESP32 RX2  <- GPS TX
#define PIN_GPS_TX   17   // ESP32 TX2  -> GPS RX
#define PIN_SD_CS     5
#define PIN_ONEWIRE   4
#define PIN_EC_A     25   // drive end A (through 10k reference resistor)
#define PIN_EC_B     26   // drive end B (direct to probe)
#define PIN_EC_SENSE 34   // node N1 between reference resistor and probe A
#define PIN_SIM_MAG  35   // simulation only
#define PIN_BUTTON   27
#define PIN_LED       2
#define PIN_BUZZER   13   // drives NPN transistor base

// ---------- Constants ----------
#define QMC_ADDR          0x0D
#define OLED_ADDR         0x3C
#define R_REF_OHM         10000.0f
#define VCC_MV            3300.0f
#define MAG_SAMPLES       20
#define EC_REPEATS        8
#define NT_PER_LSB        33.33f   // QMC5883L +/-8 G range: 3000 LSB per gauss
#define MAX_HDOP          3.0f
#define EC_TEMP_COEFF     0.02f    // 2 % per degC, standard soil approximation

Adafruit_SSD1306 oled(128, 64, &Wire, -1);
TinyGPSPlus gps;
HardwareSerial gpsSerial(2);
OneWire oneWire(PIN_ONEWIRE);
DallasTemperature tempSensor(&oneWire);

bool sdOk = false;
bool magOk = false;
uint32_t stationId = 0;
uint32_t lastDisplay = 0;
float liveBtotal = NAN;

struct Reading {
  float bx, by, bz, btotal;   // nT
  float rsOhm, ecUs, ec25Us;  // ohm, microsiemens
  float tempC;
};

// ---------- Beeper ----------
void beep(uint16_t ms) {
  digitalWrite(PIN_BUZZER, HIGH);
  delay(ms);
  digitalWrite(PIN_BUZZER, LOW);
}

// ---------- QMC5883L magnetometer ----------
bool qmcWrite(uint8_t reg, uint8_t val) {
  Wire.beginTransmission(QMC_ADDR);
  Wire.write(reg);
  Wire.write(val);
  return Wire.endTransmission() == 0;
}
```

```
bool qmcInit() {
  if (!qmcWrite(0x0B, 0x01)) return false;   // set/reset period
  return qmcWrite(0x09, 0x1D);               // 512 OSR, 8 G, 200 Hz, continuous
}

bool qmcRead(int16_t &x, int16_t &y, int16_t &z) {
  Wire.beginTransmission(QMC_ADDR);
  Wire.write(0x06);                          // status register
  if (Wire.endTransmission(false) != 0) return false;
  Wire.requestFrom((uint8_t)QMC_ADDR, (uint8_t)1);
  if (!Wire.available()) return false;
  uint8_t status = Wire.read();
  if (!(status & 0x01)) return false;        // data not ready

  Wire.beginTransmission(QMC_ADDR);
  Wire.write(0x00);
  if (Wire.endTransmission(false) != 0) return false;
  Wire.requestFrom((uint8_t)QMC_ADDR, (uint8_t)6);
  if (Wire.available() < 6) return false;
  x = (int16_t)(Wire.read() | (Wire.read() << 8));
  y = (int16_t)(Wire.read() | (Wire.read() << 8));
  z = (int16_t)(Wire.read() | (Wire.read() << 8));
  return true;
}

// Average N magnetometer samples. Returns false if the sensor does not answer.
bool readMagnetometer(float &bx, float &by, float &bz, float &bt, uint8_t n) {
#if SIM_MODE
  int raw = analogRead(PIN_SIM_MAG);                      // 0..4095
  bt = 30000.0f + ((raw / 4095.0f) - 0.5f) * 10000.0f;    // 25000..35000 nT
  bx = bt; by = 0; bz = 0;
  return true;
#else
  if (!magOk) return false;
  float sx = 0, sy = 0, sz = 0;
  uint8_t got = 0;
  uint32_t t0 = millis();
  while (got < n && millis() - t0 < 1500) {
    int16_t x, y, z;
    if (qmcRead(x, y, z)) {
      sx += x; sy += y; sz += z;
      got++;
    }
    delay(6);
  }
  if (got == 0) return false;
  bx = (sx / got) * NT_PER_LSB;
  by = (sy / got) * NT_PER_LSB;
  bz = (sz / got) * NT_PER_LSB;
  bt = sqrtf(bx * bx + by * by + bz * bz);
  return true;
#endif
}

// ---------- Soil resistance (alternating polarity) ----------
float readSenseMv() {
  uint32_t acc = 0;
  for (int i = 0; i < 16; i++) acc += analogReadMilliVolts(PIN_EC_SENSE);
  return acc / 16.0f;
}

float measureSoilResistance() {
#if SIM_MODE
  float v = readSenseMv();
  if (v < 20) v = 20;
  if (v > VCC_MV - 20) v = VCC_MV - 20;
  return R_REF_OHM * v / (VCC_MV - v);
#else
  float sum = 0;
  int count = 0;
  for (int i = 0; i < EC_REPEATS; i++) {
    // Phase A: A high, B low. N1 = Vcc * Rs / (Rref + Rs)
    pinMode(PIN_EC_A, OUTPUT); pinMode(PIN_EC_B, OUTPUT);
    digitalWrite(PIN_EC_A, HIGH); digitalWrite(PIN_EC_B, LOW);
    delay(2);
    float va = readSenseMv();
    // Phase B: A low, B high. N1 = Vcc * Rref / (Rref + Rs)
    digitalWrite(PIN_EC_A, LOW); digitalWrite(PIN_EC_B, HIGH);
    delay(2);
    float vb = readSenseMv();
    // Probes idle (no DC across the soil)
    digitalWrite(PIN_EC_B, LOW);
    pinMode(PIN_EC_A, INPUT); pinMode(PIN_EC_B, INPUT);

    // Keep readings inside the ADC's usable range (about 150 mV to 3.1 V)
    if (va > 150 && va < 3100) { sum += R_REF_OHM * va / (VCC_MV - va); count++; }
    if (vb > 150 && vb < 3100) { sum += R_REF_OHM * (VCC_MV - vb) / vb; count++; }
  }
  if (count == 0) return NAN;   // open circuit (dry sand) or short
  return sum / count;
#endif
}

// ---------- Full station measurement ----------
bool takeMeasurement(Reading &r) {
  if (!readMagnetometer(r.bx, r.by, r.bz, r.btotal, MAG_SAMPLES)) return false;
```

```
  r.rsOhm = measureSoilResistance();
  tempSensor.requestTemperatures();
  r.tempC = tempSensor.getTempCByIndex(0);
  if (r.tempC < -50 || r.tempC > 100) r.tempC = NAN;
  r.ecUs = isnan(r.rsOhm) ? NAN : 1.0e6f / r.rsOhm;     // relative conductance index
  if (isnan(r.ecUs) || isnan(r.tempC)) r.ec25Us = NAN;
  else r.ec25Us = r.ecUs / (1.0f + EC_TEMP_COEFF * (r.tempC - 25.0f));
  return true;
}

// ---------- Position ----------
bool getPosition(double &lat, double &lon, float &hdop, uint8_t &sats, char *stamp, size_t n) {
#if SIM_MODE
  static double simLat = -24.6282, simLon = 25.9231;   // Gaborone area
  simLat += 0.00009; simLon += ((stationId % 2) ? 0.00009 : -0.00009);
  lat = simLat; lon = simLon; hdop = 1.2f; sats = 9;
  snprintf(stamp, n, "2026-10-13T08:%02u:00Z", (unsigned)(stationId % 60));
  return true;
#else
  bool fix = gps.location.isValid() && gps.location.age() < 3000;
  lat = fix ? gps.location.lat() : 0.0;
  lon = fix ? gps.location.lng() : 0.0;
  hdop = gps.hdop.isValid() ? gps.hdop.hdop() : 99.9f;
  sats = gps.satellites.isValid() ? gps.satellites.value() : 0;
  if (gps.date.isValid() && gps.time.isValid()) {
    snprintf(stamp, n, "%04u-%02u-%02uT%02u:%02u:%02uZ", gps.date.year(), gps.date.month(),
             gps.date.day(), gps.time.hour(), gps.time.minute(), gps.time.second());
  } else {
    snprintf(stamp, n, "unknown");
  }
  return fix && hdop <= MAX_HDOP;
#endif
}

// ---------- SD logging ----------
void ensureHeader() {
  if (!SD.exists("/survey.csv")) {
    File f = SD.open("/survey.csv", FILE_WRITE);
    if (f) {
      f.println("station,utc,lat,lon,hdop,sats,bx_nT,by_nT,bz_nT,btotal_nT,rs_ohm,ec_uS,temp_C,ec25_uS,fix_ok");
      f.close();
    }
  }
}

uint32_t countStations() {
  uint32_t n = 0;
  File f = SD.open("/survey.csv", FILE_READ);
  if (!f) return 0;
  while (f.available()) { if (f.read() == '\n') n++; }
  f.close();
  return n > 0 ? n - 1 : 0;   // minus header
}

bool logRow(const Reading &r, double lat, double lon, float hdop, uint8_t sats,
            const char *stamp, bool fixOk) {
  if (!sdOk) return false;
  File f = SD.open("/survey.csv", FILE_APPEND);
  if (!f) return false;
  f.printf("%lu,%s,%.6f,%.6f,%.1f,%u,%.0f,%.0f,%.0f,%.0f,%.0f,%.1f,%.1f,%.1f,%d\n",
           (unsigned long)stationId, stamp, lat, lon, hdop, sats,
           r.bx, r.by, r.bz, r.btotal, r.rsOhm, r.ecUs, r.tempC, r.ec25Us, fixOk ? 1 : 0);
  f.close();
  return true;
}

// ---------- OLED ----------
void showStatus() {
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0, 0);
  oled.printf("Stn %lu  SD:%s", (unsigned long)stationId, sdOk ? "ok" : "ERR");
  oled.setCursor(0, 12);
#if SIM_MODE
  oled.print("GPS: simulated");
#else
  if (gps.location.isValid()) oled.printf("GPS fix  sats:%u", gps.satellites.value());
  else oled.print("GPS: searching...");
#endif
  oled.setCursor(0, 24);
  if (!isnan(liveBtotal)) oled.printf("|B| %.0f nT", liveBtotal);
  else oled.print("|B| --");
  oled.setCursor(0, 40);
  oled.print("Press button to log");
  oled.display();
}

void showResult(const Reading &r, bool fixOk) {
  oled.clearDisplay();
  oled.setCursor(0, 0);
  oled.printf("Saved stn %lu %s", (unsigned long)stationId, fixOk ? "" : "(no fix)");
  oled.setCursor(0, 14);
  oled.printf("B  %.0f nT", r.btotal);
  oled.setCursor(0, 26);
  if (isnan(r.rsOhm)) oled.print("Rs open circuit");
```

```
  else oled.printf("Rs %.0f ohm", r.rsOhm);
  oled.setCursor(0, 38);
  if (isnan(r.tempC)) oled.print("T  --");
  else oled.printf("T  %.1f C", r.tempC);
  oled.setCursor(0, 50);
  if (isnan(r.ec25Us)) oled.print("EC25 --");
  else oled.printf("EC25 %.0f uS", r.ec25Us);
  oled.display();
}

// ---------- Setup / loop ----------
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_EC_SENSE, ADC_11db);

  Wire.begin(PIN_SDA, PIN_SCL);
  Wire.setClock(100000);
  oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0, 0);
  oled.println("Kalahari survey node");
  oled.println("Starting...");
  oled.display();

  gpsSerial.begin(9600, SERIAL_8N1, PIN_GPS_RX, PIN_GPS_TX);
  tempSensor.begin();

#if !SIM_MODE
  magOk = qmcInit();
#else
  magOk = true;
#endif

  sdOk = SD.begin(PIN_SD_CS);
  if (sdOk) {
    ensureHeader();
    stationId = countStations();   // continue numbering after a power cycle
  }

  if (!magOk || !sdOk) { beep(600); }   // long beep = a sensor or the card failed
  else { beep(80); delay(80); beep(80); }
  showStatus();
}

void loop() {
  // Keep the GPS parser fed
  while (gpsSerial.available()) gps.encode(gpsSerial.read());

  // Live magnetic readout twice a second
  if (millis() - lastDisplay > 500) {
    lastDisplay = millis();
    float bx, by, bz, bt;
    liveBtotal = readMagnetometer(bx, by, bz, bt, 1) ? bt : NAN;
    showStatus();
  }

  // Survey button (active low, simple debounce)
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);
    if (digitalRead(PIN_BUTTON) == LOW) {
      digitalWrite(PIN_LED, HIGH);
      stationId++;

      Reading r;
      double lat, lon; float hdop; uint8_t sats; char stamp[24];
      bool fixOk = getPosition(lat, lon, hdop, sats, stamp, sizeof(stamp));
      bool measured = takeMeasurement(r);

      if (measured) {
        bool saved = logRow(r, lat, lon, hdop, sats, stamp, fixOk);
        showResult(r, fixOk);
        Serial.printf("stn %lu B=%.0f nT Rs=%.0f ohm T=%.1f C fix=%d saved=%d\n",
                      (unsigned long)stationId, r.btotal, r.rsOhm, r.tempC, fixOk, saved);
        if (saved && fixOk) beep(120);
        else { beep(80); delay(80); beep(80); delay(80); beep(80); }   // saved without fix, or SD error
      } else {
        stationId--;
        oled.clearDisplay(); oled.setCursor(0, 20); oled.print("Magnetometer error");
        oled.display();
        beep(600);
      }
      digitalWrite(PIN_LED, LOW);
      while (digitalRead(PIN_BUTTON) == LOW) { delay(10); }   // wait for release
      delay(1500);   // let the operator read the result
    }
  }
}
```

Appendix B: Wokwi diagram.json

```
{
  "version": 1,
  "author": "Montle Fredah Segomotso",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    { "type": "board-ssd1306", "id": "oled", "top": 140, "left": -180, "attrs": { "i2cAddress": "0x3c" } },
    { "type": "wokwi-ds18b20", "id": "temp1", "top": -120, "left": 220, "attrs": {} },
    { "type": "wokwi-resistor", "id": "r1", "top": -50, "left": 150, "attrs": { "value": "4700" } },
    { "type": "wokwi-microsd-card", "id": "sd1", "top": 60, "left": 260, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "potMag", "top": -140, "left": -180, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "potEC", "top": -140, "left": -80, "attrs": {} },
    { "type": "wokwi-pushbutton", "id": "btn1", "top": 200, "left": 150, "attrs": { "color": "green" } },
    { "type": "wokwi-led", "id": "led1", "top": 270, "left": 240, "attrs": { "color": "red" } },
    { "type": "wokwi-resistor", "id": "r2", "top": 250, "left": 180, "attrs": { "value": "220" } },
    { "type": "wokwi-buzzer", "id": "bz1", "top": 300, "left": 100, "attrs": { "volume": "0.1" } }
  ],
  "connections": [
    ["esp:TX", "$serialMonitor:RX", "", []],
    ["esp:RX", "$serialMonitor:TX", "", []],

    ["oled:SDA", "esp:21", "green", []],
    ["oled:SCL", "esp:22", "purple", []],
    ["oled:VCC", "esp:3V3", "red", []],
    ["oled:GND", "esp:GND.1", "black", []],

    ["temp1:DQ", "esp:4", "orange", []],
    ["temp1:VCC", "esp:3V3", "red", []],
    ["temp1:GND", "esp:GND.1", "black", []],
    ["r1:1", "esp:3V3", "red", []],
    ["r1:2", "esp:4", "orange", []],

    ["sd1:CS", "esp:5", "blue", []],
    ["sd1:SCK", "esp:18", "yellow", []],
    ["sd1:DO", "esp:19", "green", []],
    ["sd1:DI", "esp:23", "purple", []],
    ["sd1:VCC", "esp:3V3", "red", []],
    ["sd1:GND", "esp:GND.1", "black", []],

    ["potMag:SIG", "esp:35", "green", []],
    ["potMag:VCC", "esp:3V3", "red", []],
    ["potMag:GND", "esp:GND.1", "black", []],

    ["potEC:SIG", "esp:34", "blue", []],
    ["potEC:VCC", "esp:3V3", "red", []],
    ["potEC:GND", "esp:GND.1", "black", []],

    ["btn1:1.l", "esp:27", "green", []],
    ["btn1:2.l", "esp:GND.1", "black", []],

    ["esp:2", "r2:1", "red", []],
    ["r2:2", "led1:A", "red", []],
    ["led1:C", "esp:GND.1", "black", []],

    ["esp:13", "bz1:2", "orange", []],
    ["bz1:1", "esp:GND.1", "black", []]
  ],
  "dependencies": {}
}
```

Appendix C: Python mapping script (map_survey.py)

Requires numpy, pandas and matplotlib.

```
#!/usr/bin/env python3
"""
Kalahari Magnetic and Soil Survey Node - data processing

Usage:
    python map_survey.py survey.csv            # process a real survey file
    python map_survey.py --demo                # synthetic data with a buried dipole

Outputs (in the current folder):
    survey_maps.png     magnetic anomaly map and soil conductivity map
    anomalies.csv       stations flagged as anomalous
"""
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

Z_THRESHOLD = 2.0   # flag stations more than 2 standard deviations from the trend


def make_demo(path="demo_survey.csv", seed=7):
    """Synthetic grid survey over Gaborone with one buried magnetic body.
    Illustrative only: not real field data."""
    rng = np.random.default_rng(seed)
    lat0, lon0 = -24.6282, 25.9231
    xs, ys = np.meshgrid(np.arange(0, 200, 10), np.arange(0, 200, 10))
    x, y = xs.ravel(), ys.ravel()
    lat = lat0 + y / 111_320.0
    lon = lon0 + x / (111_320.0 * np.cos(np.radians(lat0)))
    r2 = (x - 120) ** 2 + (y - 90) ** 2
    h = 25.0
    dipole = 900.0 * (2 * h**2 - r2) / (r2 + h**2) ** 2.5 * h**3     # nT, illustrative
    regional = 30000 + 0.8 * x - 0.5 * y                               # gentle gradient
    btotal = regional + dipole + rng.normal(0, 25, x.size)
    conductive = 1.0 / (1 + np.exp(-(120 - x) / 15)) * 0.0 + np.exp(-r2 / (2 * 30**2))
    rs = 40000 / (1 + 3.5 * conductive) * rng.normal(1, 0.06, x.size)
    temp = rng.normal(27, 0.6, x.size)
    ec = 1e6 / rs
    ec25 = ec / (1 + 0.02 * (temp - 25))
    df = pd.DataFrame({
        "station": np.arange(1, x.size + 1), "utc": "demo", "lat": lat, "lon": lon,
        "hdop": 1.2, "sats": 9, "bx_nT": btotal, "by_nT": 0, "bz_nT": 0,
        "btotal_nT": btotal, "rs_ohm": rs, "ec_uS": ec, "temp_C": temp,
        "ec25_uS": ec25, "fix_ok": 1,
    })
    df.to_csv(path, index=False)
    return path


def to_local_metres(df):
    lat0, lon0 = df["lat"].mean(), df["lon"].mean()
    df["x_m"] = (df["lon"] - lon0) * 111_320.0 * np.cos(np.radians(lat0))
    df["y_m"] = (df["lat"] - lat0) * 111_320.0
    return df


def remove_trend(x, y, v):
    """Fit and subtract a plane (regional field) from v."""
    a = np.column_stack([x, y, np.ones_like(x)])
    coef, *_ = np.linalg.lstsq(a, v, rcond=None)
    return v - a @ coef


def main():
    if len(sys.argv) < 2:
        print(__doc__)
        sys.exit(1)
    path = make_demo() if sys.argv[1] == "--demo" else sys.argv[1]

    df = pd.read_csv(path)
    df = df[df["fix_ok"] == 1].dropna(subset=["btotal_nT"]).copy()
    if len(df) < 10:
        sys.exit("Need at least 10 stations with a GPS fix.")
    df = to_local_metres(df)

    df["b_anom_nT"] = remove_trend(df["x_m"].values, df["y_m"].values, df["btotal_nT"].values)
    df["b_z"] = (df["b_anom_nT"] - df["b_anom_nT"].mean()) / df["b_anom_nT"].std()
    ec = df["ec25_uS"].fillna(df["ec_uS"])
    df["ec_z"] = (ec - ec.mean()) / ec.std()
    df["flag"] = (df["b_z"].abs() > Z_THRESHOLD) | (df["ec_z"] > Z_THRESHOLD)

    df[df["flag"]].to_csv("anomalies.csv", index=False)
    print(f"{len(df)} stations processed, {int(df['flag'].sum())} flagged -> anomalies.csv")

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.6), constrained_layout=True)
    lim = np.nanmax(np.abs(df["b_anom_nT"]))
    s1 = axes[0].scatter(df["x_m"], df["y_m"], c=df["b_anom_nT"], cmap="RdBu_r",
                         vmin=-lim, vmax=lim, s=90, edgecolor="k", linewidth=0.3)
    axes[0].set_title("Magnetic anomaly (trend removed)")
```

```
    fig.colorbar(s1, ax=axes[0], label="nT")
    s2 = axes[1].scatter(df["x_m"], df["y_m"], c=ec, cmap="viridis", s=90,
                         edgecolor="k", linewidth=0.3)
    axes[1].set_title("Soil conductance index (temperature corrected)")
    fig.colorbar(s2, ax=axes[1], label="uS")
    for ax in axes:
        flagged = df[df["flag"]]
        ax.scatter(flagged["x_m"], flagged["y_m"], s=170, facecolors="none",
                   edgecolors="black", linewidths=1.4)
        ax.set_xlabel("Easting (m)")
        ax.set_ylabel("Northing (m)")
        ax.set_aspect("equal")
    fig.suptitle("Kalahari survey node - station map (ringed = flagged)")
    fig.savefig("survey_maps.png", dpi=160)
    print("Saved survey_maps.png")


if __name__ == "__main__":
    main()
```